A Feature-Rich Embedded NIDS with eBPF/XDP: Detector and Architecture Trade-offs

Shiqi Wu[1,3 [0009−0003−4677−7958]⋆], Oleksii Koshovyi[2,3 [0009−0009−2027−8652]], Georgios Pseiridis Pseiras[3 [0009−0001−3007−9923]], Victor Morel[1 [0000−0001−9482−8906]], and Romaric Duvignau[1 [0000−0003−1268−9311]]

[1] Chalmers University of Technology, Gothenburg, Sweden
shiqiw@student.chalmers.se, {morelv,duvignau}@chalmers.se
[2] University of Gothenburg, Gothenburg, Sweden
guskoshol@student.gu.se
[3] Ericsson AB, Gothenburg, Sweden
{shiqi.wu,oleksii.koshovyi,georgios.pseiridis.pseiras}@ericsson.com

Abstract. Distributed Denial-of-Service (DDoS) attacks remain a significant threat to transport networks, creating a need for accurate network intrusion detection on resource-constrained operational platforms. This paper presents a flow-based Network Intrusion Detection System (NIDS) that combines an Isolation Forest detector with eBPF/XDP-based traffic filtering at the kernel level. We evaluate three deployment architectures – a monolithic implementation and two microservice-based implementations using gRPC and Kafka – to determine how software architecture affects detection quality, latency, and deployment overhead. Experiments on a Raspberry Pi 5 testbed, using the CIC-DDoS2019 dataset replayed as live network traffic, show that detector choice has the greatest influence on detection accuracy, while the communication architecture can materially affect detector-side performance. The Isolation Forest detects substantially more attack windows than a statistical baseline and achieves an F1 score of 0.965 in the monolithic deployment. The gRPC implementation achieves similar detection performance while adding less than 1 ms of mean transport overhead per analysis window. In the evaluated Kafka implementation, mean transport overhead is approximately 28 ms and the F1 score is about five percentage points below the monolithic result. These findings quantify the accuracy and latency trade-offs involved in deploying practical NIDS architectures on resource-constrained transport-network infrastructure.

Keywords: Network intrusion detection · DDoS detection · eBPF/XDP · Isolation Forest · Microservices · Edge deployment

⋆ S. Wu and O. Koshovyi contributed equally to this work and share first authorship.

1 Introduction

Distributed Denial-of-Service (DDoS) attacks remain a persistent threat to network infrastructure. Recent incidents have exceeded 30 Tbps, approximately an order of magnitude larger than attacks reported two years earlier, with telecommunications infrastructure among the main targets [4]. As transport networks carry increasingly critical services, intrusion detection must identify attacks accurately while operating under the throughput and resource constraints of network infrastructure.

Network Intrusion Detection Systems (NIDS) are well suited to this setting because they observe aggregated traffic at network vantage points and can trigger mitigation before downstream services become saturated. Flow-based NIDS summarize packets that share a source address, destination address, ports, and protocol into compact statistical representations. These representations capture traffic behaviour more effectively than individual packet counts and can be analysed over consecutive time windows to support online detection [13,6]. However, practical deployment still presents two challenges.

First, detection methods based on a limited set of traffic statistics, such as packet counts alone, may fail to distinguish low-volume or evolving attacks from legitimate traffic variation. Richer flow-level features can improve this distinction, but the resulting detector must remain lightweight enough to run close to the network edge. We therefore use an Isolation Forest (IF) [14], an unsupervised anomaly-detection algorithm that isolates unusual observations through randomized partitioning. Unlike supervised approaches, IF does not require labelled attack samples during training and may therefore detect previously unseen traffic patterns. Its relatively low training and inference costs also make it suitable for resource-constrained platforms [22,17].

Second, detection alone does not protect the network unless identified traffic can be filtered efficiently. The Extended Berkeley Packet Filter (eBPF) and Express Data Path (XDP) allow packet-processing programs to execute within the Linux kernel and discard packets before they traverse the conventional network stack [12]. Integrating detection with eBPF/XDP therefore enables mitigation close to the network ingress while limiting the processing and memory costs imposed by attack traffic.

The architecture connecting traffic monitoring, detection, and mitigation introduces an additional operational trade-off. A monolithic implementation minimizes communication overhead but couples the system components and limits independent deployment and scaling. A microservice architecture improves modularity and allows components to be deployed and scaled separately, but inter-service communication may increase latency or alter the timing and completeness of the traffic windows supplied to the detector. This architectural effect is often considered separately from detection accuracy, although both determine whether a NIDS is viable in an operational environment.

To study this trade-off, we implement a flow-based NIDS that combines an IF detector using an extended set of flow features with eBPF/XDP-based packet filtering. We compare three deployment architectures: a monolithic implemen-

tation, a synchronous microservice implementation using gRPC, and an asynchronous implementation using Apache Kafka. gRPC provides direct request–response communication using Protocol Buffers, whereas Kafka decouples producers and consumers through persistent brokered messaging. These alternatives represent distinct latency, buffering, scalability, and failure-handling characteristics.

We evaluate the implementations on a Raspberry Pi 5 testbed by replaying labelled traffic from the CIC-DDoS2019 dataset [19] as live network traffic. The dataset contains both reflection-based and exploitation-based DDoS attacks and provides a broad set of flow-level features. Although its synthetically generated traffic does not fully represent an operational transport network, it enables controlled and repeatable comparison of detection and deployment behaviour.

1.1 Research Questions

We investigate the following research questions:

RQ1: How does the selected feature-based Isolation Forest compare with a statistical baseline on a resource-constrained transport-network platform?

RQ2: How do monolithic, Kafka-based, and gRPC-based deployment architectures affect the detection quality and operational performance of a NIDS?

To answer these questions, we develop a NIDS that combines flow-based anomaly detection with eBPF/XDP filtering at the network interface. The system uses an Isolation Forest trained on flow features extracted by GoFlowMeter[4], our open-source Go implementation of CICFlowMeter [13]. We deploy the system using monolithic, Kafka-based, and gRPC-based architectures and compare their detection quality, processing latency, and resource consumption on a Raspberry Pi 5 testbed. For controlled and repeatable evaluation, more than 150 GB of traffic from the CIC-DDoS2019 dataset [19] is replayed through a physical network interface.

1.2 Contributions

This paper makes the following contributions:

- We design and implement a flow-based NIDS that integrates Isolation Forest detection with eBPF/XDP packet filtering, enabling live traffic to be monitored and malicious packets to be discarded at the network interface before traversing the conventional kernel network stack.
- We implement and empirically compare monolithic, Kafka-based, and gRPC-based deployment architectures on a resource-constrained edge platform. The comparison quantifies their effects on detection quality, detector-side latency, and engine-side heap allocation, showing that the gRPC architecture introduces substantially less communication overhead than the Kafka architecture.

[4] The GitHub repository will be released as open source upon publication.

- We develop GoFlowMeter, an open-source Go implementation of CICFlowMeter [13], which extracts 80-dimensional flow representations for Isolation Forest detection. GoFlowMeter enables the feature-rich detector to be embedded directly in the Go-based NIDS and supports a controlled comparison with packet-count-only Isolation Forest input.
- We develop a standalone traffic-replay tool that sequentially replays more than 150 GB of CIC-DDoS2019 traffic through a physical network interface, supporting controlled and reproducible evaluation of the complete online detection pipeline.

1.3 Paper Organization

The remainder of this paper is organized as follows. Section 2 reviews related work on flow-based DDoS detection, eBPF/XDP integration, and NIDS deployment architectures. Section 3 presents the NIDS design, feature-extraction pipeline, deployment architectures, and kernel-level filtering mechanism. Section 4 describes the experimental methodology and reports the hyperparameter-selection, detection-quality, and architectural-performance results, followed by their operational implications and limitations. Section 5 discusses the operational implications and limitations. Finally, Section 6 summarizes the findings and directions for future work.

2 Related Work

We review related work in three areas: flow-based DDoS detection, integration of machine learning with eBPF/XDP, and the operational effects of alternative NIDS deployment architectures.

2.1 Flow-Based DDoS Detection

DDoS detection methods range from lightweight statistical techniques to supervised and unsupervised machine learning. Wickman and Rygaard [22] developed a resource-efficient NIDS combining probabilistic data structures with anomaly detection. Their statistical approach used Count-Min Sketch to estimate per-source packet frequencies and HeavyKeeper to identify sources responsible for disproportionate traffic volumes. They also evaluated Isolation Forest (IF) detectors using packet count as their sole feature. On the evaluated datasets, the statistical method achieved an accuracy of 96.09%, whereas the single-feature IF achieved 73.63%. We therefore use their statistical method as the *Baseline* in the remainder of this paper and investigate whether a richer flow-level representation can make unsupervised detection more effective.

Several studies report high DDoS detection performance using supervised learning. Shaaban et al. [18] transformed one-dimensional network features into two-dimensional representations for classification with a Convolutional Neural Network (CNN). Akgun et al. [1] combined preprocessing, feature selection, and an inception-style CNN on CIC-DDoS2019. Recurrent models have also been

used to capture temporal dependencies: Shurman et al. [20] applied a Long Short-Term Memory (LSTM) network to reflection-based DDoS detection, while Djama et al. [5] combined convolutional and recurrent layers. Bhawsar et al. [3] compared Random Forest (RF) and Support Vector Machine classifiers, finding that RF reduced false negatives but required substantially longer execution time.

Although these studies report high classification scores, their results are not directly comparable because they use different datasets, preprocessing pipelines, feature sets, and evaluation protocols. More importantly for operational deployment, supervised methods depend on representative labelled attack data. Obtaining and maintaining such labels can be costly, while changing traffic patterns and previously unseen attacks can reduce the relevance of statically trained models.

Unsupervised approaches instead model normal traffic or isolate anomalous observations. Vinothina et al. [21] combined an autoencoder with XGBoost-based feature selection and detected anomalies through reconstruction error. Ghani et al. [9] used IF as an initial anomaly detector before supervised classification. These studies demonstrate the potential of unsupervised detection, but do not evaluate whether a feature-rich IF can outperform a lightweight statistical detector within a complete online NIDS running on a resource-constrained platform.

2.2 Machine Learning with eBPF/XDP

Farasat et al. [8] integrated machine-learning-based intrusion detection with eBPF/XDP packet processing. Their framework used CICFlowMeter to extract flow features and SelectKBest to retain 20 features. They compared Support Vector Machine, Decision Tree, LSTM, and Bidirectional LSTM classifiers on a generated dataset [7]; the Bidirectional LSTM produced the strongest reported detection results.

This work demonstrates that eBPF/XDP can connect machine-learning detection with kernel-level packet filtering. Its primary focus, however, is the comparison of supervised classifiers. It does not investigate unsupervised detection on a constrained edge platform or quantify how the software architecture connecting monitoring, detection, and mitigation affects end-to-end behaviour.

2.3 Operational Deployment Architectures

A NIDS may be implemented as a single process or decomposed into independently deployable services. A monolithic design avoids network communication between components, whereas microservice implementations introduce serialization and communication overhead in exchange for modularity and independent scaling. Brokered messaging systems such as Kafka provide asynchronous communication, buffering, and persistence [2]. In contrast, gRPC provides synchronous request–response communication using Protocol Buffers [10,15].

These communication models may affect more than processing latency. Buffering, polling, batching, and synchronization can influence when traffic windows

become available to the detector and, consequently, the behaviour of the complete detection pipeline. Nevertheless, the reviewed NIDS studies primarily compare detection algorithms and report classifier-level metrics. They do not compare monolithic, broker-based, and synchronous microservice architectures under the same traffic, detector, hardware, and measurement procedure.

Our work addresses these two gaps jointly. First, it evaluates whether an IF trained on a rich flow-level feature set improves detection over a lightweight statistical baseline. Second, it compares monolithic, Kafka-based, and gRPC-based implementations of the complete NIDS in terms of detection quality, latency, and resource consumption on a resource-constrained testbed.

3 NIDS Design and Deployment Architectures

This section presents the design and implementation of the proposed NIDS. The system combines flow-level feature extraction and adaptive anomaly detection with eBPF/XDP-based packet filtering. We implement the same detection pipeline using three deployment architectures—monolithic, Kafka-based, and gRPC-based—to isolate the effects of inter-component communication on detection and operational performance.

3.1 Detection Pipeline and Feature Extraction

Incoming packet metadata is grouped into fixed-length analysis windows and processed at two levels. The window-level detector determines whether the aggregate traffic is anomalous, while the per-source detector identifies the addresses contributing to the anomaly. Sources that exceed the detector-derived rate threshold are added to an eBPF blocklist and filtered in subsequent windows.

The system supports two detection approaches. The statistical baseline uses an interquartile-range-based threshold derived from packet counts (as in [22]). The machine-learning detector uses Isolation Forest with either the packet count alone or a richer flow-level representation. Following the initial warm-up period, the model is updated with observations classified as benign so that it can adapt to changes in normal traffic. Updates are suspended during detected attacks to reduce the risk of incorporating malicious observations into the model.

GoFlowMeter CICFlowMeter, a network flowmeter for extracting the features from captured network packets, is implemented in Java and cannot be embedded directly into the Go-based NIDS. We therefore implement GoFlowMeter, a native Go library that assembles packets into bidirectional flows and computes the 80-feature schema used by CICFlowMeter [13,6].

GoFlowMeter produces one feature vector for each bidirectional flow, whereas the detector requires one vector per analysis window. Let F_w denote the set of flows observed in window w, and let $x_f \in \mathbb{R}^{80}$ be the feature vector of flow f. The corresponding window-level vector $\bar{x}_w$ is the element-wise arithmetic mean

$$\bar{x}_{w,k} = \frac{1}{|F_w|} \sum_{f \in F_w} x_{f,k}, \qquad k = 1, \ldots, 80. \tag{1}$$

Undefined or infinite feature values, such as rates computed for zero-duration flows, are replaced by zero. A window containing no completed flows is represented by an 80-dimensional zero vector. Per-source representations are computed using the same aggregation rule, restricted to flows associated with the corresponding source address.

3.2 Deployment Architectures

All three implementations execute the same feature-extraction, detection, adaptation, and mitigation logic. They differ only in how the NIDS engine communicates with the detector. The architectures are shown in Figure 1.

Monolithic Architecture In the monolithic implementation, packet ingestion, window management, feature extraction, detection, mitigation, and output generation execute within one Go process. Completed feature vectors are passed directly to the detector in memory, without serialization or inter-process communication. This implementation provides the reference point for measuring the overhead introduced by service decomposition.

Kafka-Based Architecture The Kafka implementation separates the engine and detector into independent processes connected through a Kafka broker. For each completed window, the engine serializes the window identifier, aggregate features, per-source features, packet counts, and detector configuration into a message and publishes it to a feature topic. The detector consumes the message, performs classification, and publishes the result to a separate result topic. The engine consumes results asynchronously and associates them with the corresponding windows through their identifiers.

This design decouples feature production from detection and provides buffering and persistence.

gRPC-Based Architecture The gRPC implementation also separates the engine and detector, but uses a synchronous unary remote procedure call. The engine sends the completed window representation to the detector through a `Detect` request. The response contains the window-level decision, anomaly score, detection threshold, identified sources, and detector execution time.

Because the request blocks until the response is returned, the result is available within the same processing cycle as the corresponding window. The implementation avoids broker persistence and polling, but couples engine progress to detector response time.

All three implementations use the same detector code and feature representations. Consequently, observed differences arise from process separation and communication behaviour rather than from different detection algorithms.

3.3 Kernel-Level Filtering with eBPF/XDP

The NIDS uses eBPF/XDP to capture packet metadata and enforce detector decisions at the monitored network interface. The eBPF program is written in

C, loaded by the Go engine, and attached to the interface through an XDP hook. The architecture of NIDS with eBPF/XDP is shown in Figure 2.

For each incoming packet, the XDP program extracts the source address and queries a shared BPF blocklist map. If the address is present, the program returns `XDP_DROP`, discarding the packet before it traverses the conventional Linux networking stack. Otherwise, it returns `XDP_PASS` and forwards the metadata required for flow construction—including addresses, ports, protocol, TCP flags, direction, timestamp, and packet length—to user space through a ring buffer.

In user space, the engine processes these events and supplies them to the windowing and feature-extraction pipeline. After a source is identified as malicious, the engine inserts its address into the blocklist map. Subsequent packets from that source are then discarded directly by the XDP program.

4 Experimental Evaluation

The evaluation addresses the research questions introduced in Section 1.1. RQ1 regards the detection effectiveness of the feature-based Isolation Forest relative to the statistical baseline. RQ2 assesses the effects of monolithic, Kafka-based, and gRPC-based architectures on detection quality, latency, and resource consumption.

4.1 Testbed

All experiments run on a dedicated Raspberry Pi 5 equipped with a 2.4 GHz quad-core Arm Cortex-A76 processor and 16 GB of RAM, and a 64-bit Linux [16]. The platform represents a resource-constrained edge device and makes the processing and memory costs of the alternative architectures visible.

All processes belonging to a configuration are colocated on the same device. Inter-process communication uses the loopback interface. The experiments therefore measure process-separation and framework overhead without introducing variability from an external network; they do not model a geographically distributed deployment.

4.2 Traffic Replay

We evaluate the complete online pipeline by replaying the selected dataset (CIC-DDoS2019 [19]) through a physical network interface. The dataset contains more than 150 GB of traffic distributed across approximately 1,000 PCAP files.

A standalone sequential replayer transmits the files to the NIDS device through a raw socket. To avoid loading the entire dataset into memory, it reads the traffic incrementally and preloads the next PCAP while transmitting the current one. The replayer preserves the original inter-packet timing by comparing elapsed replay time with the packet timestamps and delaying transmission when necessary. Ethernet headers are rewritten to direct the frames to the monitored interface, while the original IP addresses and transport-layer ports are retained.

During the detector warm-up period, the evaluation replayer supplies traffic labelled as benign so that the initial model and statistical thresholds are not trained on attack traffic. This use of dataset labels is restricted to experimental initialization and is not part of the deployed detection path. With inter-packet

delays disabled, the replayer sustains approximately 3 Gbps, exceeding the peak rate in the evaluated traces.

4.3 Window-Level Ground Truth

CIC-DDoS2019 provides labels for complete network flows, while the NIDS classifies fixed-length analysis windows. We therefore derive window-level ground truth from the temporal intervals of the labelled flows.

Let $\mathcal{M}$ be the union of the time intervals associated with attack-labelled flows. A window w spanning the interval $[t_w^{\text{start}}, t_w^{\text{end}}]$ is labelled malicious when

$$[t_w^{\text{start}}, t_w^{\text{end}}] \cap \mathcal{M} \neq \emptyset. \tag{2}$$

A window is labelled benign only when it does not overlap any attack interval. This rule reflects the operational objective of raising an alert whenever attack traffic is present in an analysis window, including windows that contain both malicious and benign traffic.

4.4 Isolation Forest Configuration

Isolation Forest hyperparameters are selected using an offline probe that applies the same window construction and feature aggregation as the live system. Each PCAP is parsed once, and the resulting packet counts and feature vectors are cached to avoid repeating feature extraction for every candidate configuration.

The probe varies the ensemble size, subsample size, and expected anomaly fraction. Specifically, the number of trees is selected from $50, 100, 200$, the sample size from $128, 256, 512$, and the anomaly fraction from $0.01, 0.05, 0.1, 0.2$. Each configuration is evaluated using both packet-count input and the complete 80-dimensional window representation. The selected configuration is fixed before the live architecture comparisons.

4.5 Evaluation Metrics

Detection quality is measured at the window level using precision, recall, accuracy, and F1 score. Because attack and benign windows may be imbalanced, F1 score and recall are treated as the principal detection metrics.

Operational performance is measured using the following quantities:

- *Feature-processing time*: the time required by the engine to construct the flow representation for a completed window.
- *Detector time*: the time spent inside the detector performing classification and model maintenance.
- *Post-window analysis latency*: the elapsed time between the engine dispatching a completed window and receiving its detection result.
- *Communication overhead*: post-window analysis latency minus detector time.
- *Engine heap allocation*: the Go heap allocated by the NIDS engine process, sampled once per completed window.

For the monolithic architecture, communication overhead corresponds to the cost of the in-process detector call and is expected to be negligible. For Kafka and gRPC, it includes serialisation, inter-process communication, and the framework-specific processing required to return a result.

Table 1. Selected and representative Isolation Forest probe results, reported as the mean of three runs; AF = anomaly fraction, F1 = F1-score, Acc. = accuracy.

Setting	Input	Trees	Sample	AF	F1	Acc.
Selected feature-vector config.	Feature vector	200	128	0.2	0.932	0.873
Best packet-count config.	Packet count	200	128	0.2	0.929	0.868
Lowest-F1 packet-count config. at 0.2	Packet count	50	512	0.2	0.909	0.833
Best packet-count config. at 0.1	Packet count	200	512	0.1	0.830	0.710

4.6 Experimental Procedure

The evaluation consists of three stages. First, the offline probe selects the Isolation Forest configuration used in the live experiments. Second, we compare the statistical baseline and the Isolation Forest variants to answer RQ1. Third, we compare the three deployment architectures under identical traffic, detector configurations, and hardware conditions to answer RQ2.

Each reported configuration was executed three times. Detection metrics are computed over the complete set of classified windows, while timing and memory results are reported using both central tendency and dispersion. Each live detector-architecture configuration was executed three times with time window width of one second, and 75-window warmup stage.

4.7 Hyperparameter Selection

The offline probe evaluated 72 Isolation Forest configurations: 36 configurations for each of the packet-count and feature-vector input modes. Each configuration was executed three times and ranked primarily by F1 score, with accuracy used as a secondary metric.

Table 1 reports the best configuration for each input mode together with two representative packet-count configurations. The expected anomaly fraction had the strongest effect on detection quality. Every configuration using an anomaly fraction of 0.2 outperformed those using lower values. By comparison, varying the number of trees and sample size produced only small differences once the anomaly fraction was fixed.

For feature-vector input, all nine combinations using an anomaly fraction of 0.2 achieved a mean F1 score of 0.932 and an accuracy of 0.873. We selected 200 trees and a sample size of 128 from this tied set and fixed this configuration for all subsequent live experiments. The best packet-count configuration achieved an F1 score of 0.929, only 0.003 below the feature-vector result.

The strong performance of an anomaly fraction of 0.2 reflects the attack-heavy composition of the evaluated traffic and should not be interpreted as a generally optimal value for operational traffic. Moreover, because the same dataset was used for configuration selection and live evaluation, the reported results characterise performance on CIC-DDoS2019 rather than generalisation to an independent traffic distribution.

4.8 RQ1: Detection Effectiveness

We evaluate the selected Isolation Forest configuration within the live NIDS and compare it with the statistical baseline. Each architecture processes the same

Table 2. Live window-level detection quality over 1,533 analysis windows (1,455 out of 1,533 are malicious, and 26 of the malicious are inside the warm-up stage).

Architecture	Detector	Accuracy	Precision	Recall	F1
Monolithic	Baseline	0.4047	1.0000	0.3964	0.5677
Kafka	Baseline	0.4060	1.0000	0.3957	0.5670
gRPC	Baseline	0.4067	1.0000	0.3972	0.5686
Monolithic	Isolation Forest	0.9335	0.9963	0.9356	0.9650
Kafka	Isolation Forest	0.8422	0.9992	0.8409	0.9132
gRPC	Isolation Forest	0.9150	0.9977	0.9151	0.9546

sequence of 1,533 labelled analysis windows. Unlike the offline probe, the live experiment includes detector warm-up, online model updates, inter-component communication, and mitigation-state management.

Table 2 reports the resulting detection metrics. The statistical baseline achieves nearly perfect precision in all three architectures but detects fewer than 40% of malicious windows. Its F1 score therefore remains between 0.5670 and 0.5686. The low recall indicates that the baseline rarely produces false alarms but misses a substantial proportion of the attack windows.

The Isolation Forest substantially improves recall while retaining precision above 0.996 in every architecture. In the monolithic deployment, recall increases from 0.3964 to 0.9356 and the F1 score from 0.5677 to 0.9650. These correspond to improvements of 53.92 and 39.73 percentage points, respectively.

Detection results for the baseline are effectively invariant across architectures: its F1 scores differ by less than 0.002. The adaptive Isolation Forest is more sensitive to the deployment architecture. The gRPC implementation achieves an F1 score of 0.9546, 1.04 percentage points below the monolithic implementation. Kafka achieves an F1 score of 0.9132, 5.18 percentage points below the monolithic implementation. The corresponding decrease in accuracy for Kafka is 9.13 percentage points.

These results answer RQ1 for the evaluated dataset: the selected Isolation Forest provides substantially higher recall and F1 score than the packet-count-based statistical baseline while preserving high precision. However, the live comparison does not isolate the effect of the richer feature representation from the effect of replacing the statistical detector with Isolation Forest. The offline probe shows only a small difference between count-based and feature-based Isolation Forest configurations.

4.9 RQ2: Architectural Performance

We next compare the timing and memory behaviour of the three deployment architectures. Table 3 reports the mean and 99th-percentile processing, detector, and transport times for each detector–architecture combination.

The monolithic implementation introduces negligible transport overhead because the engine invokes the detector directly in memory. Its mean processing time is 0.352 ms with the baseline and 9.901 ms with Isolation Forest, showing that detector execution dominates the monolithic post-window processing cost.

Table 3. Per-window time measurements in microseconds, reported as mean / 99th percentile.

Architecture	Detector	Processing	Detector	Transport
Monolithic	Baseline	352 / 886	349 / 878	0 / 0
Monolithic	Isolation Forest	9,901 / 42,164	9,898 / 42,161	0 / 0
Kafka	Baseline	27,922 / 43,472	138 / 443	27,783 / 43,363
Kafka	Isolation Forest	44,418 / 87,870	16,231 / 59,266	28,186 / 44,484
gRPC	Baseline	789 / 2,408	143 / 434	645 / 2,096
gRPC	Isolation Forest	12,244 / 49,554	11,368 / 48,886	876 / 3,074

The gRPC implementation adds limited communication overhead. Its mean transport time is 0.645 ms with the baseline and 0.876 ms with Isolation Forest. At the 99th percentile, the corresponding values are 2.096 and 3.074 ms. Consequently, the gRPC implementation remains close to the monolithic implementation: with Isolation Forest, its mean measured processing time is 12.244 ms, compared with 9.901 ms for the monolithic implementation.

Kafka introduces substantially higher latency. Its mean transport time is 27.783 ms with the baseline and 28.186 ms with Isolation Forest. For the Isolation Forest configuration, this is approximately 32 times the mean gRPC transport time. Kafka's 99th-percentile transport time reaches 44.484 ms, compared with 3.074 ms for gRPC.

The Kafka delay is largely independent of detector complexity: its mean transport time differs by less than 0.5 ms between the baseline and Isolation Forest configurations. This indicates that broker interaction, message serialization, persistence, and consumer scheduling dominate the added latency rather than the classification algorithm itself.

Engine heap allocation. Table 4 reports the Go heap allocation of the NIDS engine, sampled once per completed window. This measurement includes neither the separate detector processes used by Kafka and gRPC nor the Kafka broker. It therefore characterizes the engine implementation rather than the total memory footprint of each architecture.

The Kafka engine has the largest measured heap allocation, averaging approximately 26.5 MB for both detectors. This increase is consistent with the buffers and background state maintained by the Kafka client. The gRPC engine has the smallest measured allocation, averaging approximately 21.2 MB, while the monolithic engine averages 22.0 MB with the baseline and 25.5 MB with Isolation Forest.

The detector choice has little effect on the Kafka and gRPC engine measurements because the model resides in a separate process. In the monolithic implementation, placing the Isolation Forest in the engine process increases its average heap allocation by approximately 3.5 MB.

Because the measurements exclude the detector services and Kafka broker, they do not support a comparison of total architecture-level memory consump-

Table 4. NIDS engine heap allocation in KB, reported as mean / maximum. Separate detector processes and the Kafka broker are excluded.

Architecture	Baseline	Isolation Forest
Monolithic	22,019 / 34,852	25,495 / 38,584
Kafka	26,595 / 46,209	26,475 / 46,389
gRPC	21,218 / 34,785	21,374 / 34,288

tion. They nevertheless show that none of the measured engine processes is individually constrained by memory on the evaluated platform.

Overall, the results answer RQ2 by showing that communication architecture materially affects both operational latency and the behaviour of the adaptive detector. gRPC remains close to the monolithic implementation in detection quality while adding less than 1 ms of mean transport overhead in the Isolation Forest configuration. Kafka provides asynchronous decoupling and buffering at the cost of approximately 28 ms of mean transport delay and lower detection quality in the evaluated implementation.

5 Discussion

5.1 Operational Implications

The results separate primarily along the detector dimension. Across all three architectures, the statistical baseline achieves near-perfect precision but recall below 0.40, indicating that it raises few false alarms but misses many labelled attack windows. Isolation Forest increases recall to between 0.841 and 0.936 while maintaining precision above 0.996. For the evaluated traffic, detector choice therefore has a substantially larger effect on detection quality than deployment architecture.

The offline probe found only a small difference between packet-count and feature-vector Isolation Forest inputs. This result is consistent with the composition of CIC-DDoS2019, which contains predominantly high-volume flooding attacks for which packet volume is already a strong indicator. The result does not establish that the additional flow features are unnecessary in other environments. Traffic containing low-rate, application-layer, or gradually developing attacks may benefit more from behavioural features, but evaluating such attacks requires a more diverse dataset.

Architecture nevertheless affects the behaviour of the complete adaptive pipeline. The statistical baseline produces almost identical results in all three deployments, whereas the Isolation Forest achieves an F1 score of 0.9650 in the monolithic implementation, 0.9546 with gRPC, and 0.9132 with Kafka. Because all variants use the same detector implementation, the Kafka difference is not caused by a different classification algorithm. One plausible explanation is that asynchronous message delivery changes the timing or ordering of classification and online model updates. The present instrumentation cannot confirm this explanation, and the observed difference should therefore be treated as an architectural effect of the implementation rather than as an inherent property of

Kafka. Establishing the cause would require tracing window identifiers, delivery order, model state, and update events throughout the pipeline.

The timing results show a clear distinction between the two microservice communication models. gRPC adds less than 1 ms of mean transport overhead with Isolation Forest and remains close to the monolithic implementation. Kafka adds approximately 28 ms. Since both microservice deployments communicate over the loopback interface, this difference reflects framework and processing overhead rather than an external network delay. Kafka provides asynchronous buffering, persistence, and producer–consumer decoupling, whereas the unary gRPC implementation follows a direct synchronous request–response path. The appropriate architecture therefore depends on whether low decision latency or asynchronous decoupling is more important to the deployment.

The monolithic implementation provides the lowest measured latency and the highest detection quality in this evaluation. This does not establish that a monolithic architecture is generally preferable. All components execute on one device, and the evaluated workload does not test independent scaling, multi-node deployment, service recovery, or sustained overload of a single process. The results instead show that service decomposition has a measurable cost and that gRPC preserves most of the monolithic performance when such decomposition is required.

The standalone replayer does not appear to constrain the experiment. With inter-packet delays disabled, it reaches an average replay rate of approximately 3 Gbps, around three times the highest rate present in the evaluated trace. The measured system behaviour is therefore unlikely to result from an inability of the replayer to supply traffic at the dataset rate.

5.2 Limitations and Validity Threats

The results should be interpreted in the light of the following limitations. First, CIC-DDoS2019 contains synthetic traffic collected in 2019 and is dominated by high-volume flooding attacks. Because it does not capture the diversity of contemporary operational traffic, changing network configurations, or low-rate and application-layer attacks, the conclusions characterize the evaluated dataset rather than general detection guarantees. Second, Isolation Forest hyperparameters were selected and evaluated on traffic from the same dataset. The live experiment differs from the offline probe through warm-up, online updates, communication, and mitigation state, yet it is not an independent test of generalization; a stronger evaluation would tune the detector on one traffic period or dataset and evaluate it on a separate, unseen distribution. Third, a window is labelled malicious whenever it overlaps a labelled attack interval. This conservative rule matches the objective of detecting any active attack, but it may penalize a detector that does not respond to a short or low-volume overlap within a window. Fourth, only two input representations were studied: packet count alone and the complete 80-feature vector. Redundant or irrelevant dimensions may reduce Isolation Forest effectiveness [11], while a selected feature subset could lower classification and serialization costs, so the evaluation compares two endpoints of the feature space rather than identifying an optimal representation.

Furthermore, the live comparison lacks a count-based variant and therefore does not isolate the contribution of the richer feature set to the improvement over the statistical baseline. Fifth, all experiments use one Raspberry Pi 5 running a general-purpose Linux operating system, where background activity, CPU-frequency changes, scheduling, and garbage collection introduce timing variation. The reported values characterize this platform and should be interpreted through their consistent relative ordering rather than as hard real-time bounds. Sixth, the timing boundaries are not fully identical across architectures: the monolithic measurement includes the complete post-window routine, whereas the Kafka and gRPC measurements begin when a request or message is emitted after feature extraction. The comparison therefore concerns detector-side post-window latency rather than complete packet-to-decision latency; a future implementation should place common timestamps at window completion and at decision application. Seventh, the memory measurements cover only the Go heap of the engine process and exclude the detector processes and the Kafka broker. They therefore support conclusions about engine-side allocation, but not a complete architecture-level memory comparison. Finally, mitigation blocks individual source addresses using per-source traffic statistics. This approach is less effective against attacks distributed across many low-rate sources and does not expire blocklist entries automatically; time-bounded blocking and detection mechanisms that reason jointly over distributed sources remain subjects for future work.

Ethics. The work is defensive in purpose and uses only the public, synthetically generated CIC-DDoS2019 dataset, which contains no personal data. All traffic replay was confined to an isolated testbed consisting of devices controlled by the researchers; no attack traffic was sent to third parties or the public Internet.

6 Conclusion

This paper presented a flow-based NIDS that combines Isolation Forest anomaly detection with eBPF/XDP packet filtering and compared monolithic, Kafka-based, and gRPC-based deployment architectures on a resource-constrained edge platform.

Regarding RQ1, Isolation Forest substantially improved detection over the statistical baseline on the evaluated CIC-DDoS2019 traffic. In the monolithic deployment, it increased recall from 0.3964 to 0.9356 and F1 score from 0.5677 to 0.9650 while maintaining precision above 0.996. The offline probe found only a small advantage for the complete flow-feature vector over packet count alone, and the current live evaluation does not isolate the contribution of the richer representation. The results therefore support the effectiveness of the selected Isolation Forest detector, but further evaluation is required to determine when the additional features provide a material advantage.

Regarding RQ2, the communication architecture affected both latency and the behaviour of the adaptive detector. The gRPC implementation remained close to the monolithic deployment, achieving an F1 score of 0.9546 while adding less than 1 ms of mean transport overhead with Isolation Forest. Kafka provided asynchronous decoupling but added approximately 28 ms of mean transport

overhead and achieved an F1 score of 0.9132. The cause of the Kafka detection difference was not isolated and requires further instrumentation.

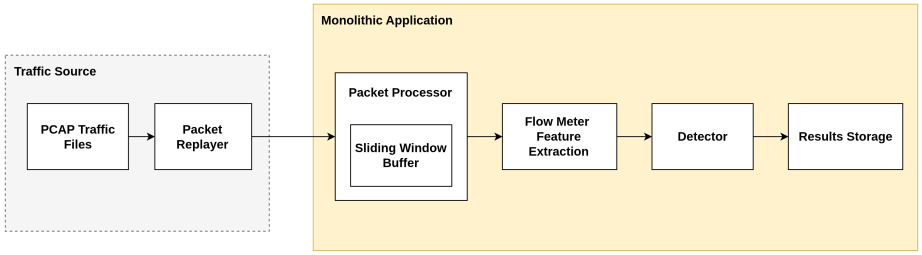
Overall, the results show that accurate anomaly detection and kernel-level mitigation can be deployed on a constrained platform, but that architectural modularity is not free. A synchronous microservice boundary can preserve near-monolithic behaviour, whereas broker-based asynchronous communication introduces trade-offs in latency and, in this implementation, detection quality. Future work should evaluate independent training and test traffic, select a smaller feature subset, measure complete architecture-level resource consumption, and test the system under more diverse and operationally representative attacks.

References

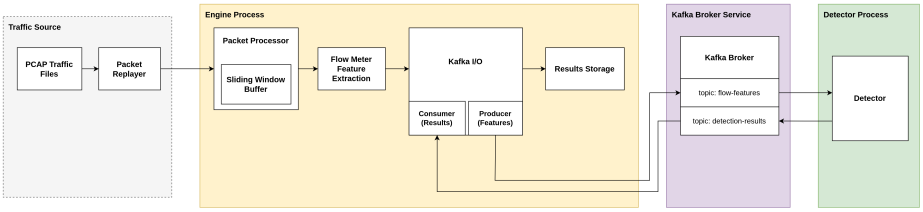
1. Akgun, D., Hizal, S., Cavusoglu, U.: A new DDoS attacks intrusion detection model based on deep learning for cybersecurity. Computers & Security **118**, 102748 (2022). https://doi.org/https://doi.org/10.1016/j.cose.2022.102748
2. Apache Software Foundation: Apache kafka (05 2026), https://kafka.apache.org/, accessed: 2026-05-05
3. Bhawsar, P., Kumar, A., Kumar, S., Gupta, V., Rana, R., Mehta, P., Mishra, Z.: Machine learning-based detection of ddos attacks: An evaluation of svm and random forest approaches. In: 2025 13th International Conference on Intelligent Systems and Embedded Design (ISED). pp. 61–67 (2025). https://doi.org/10.1109/ISED67359.2025.11405254
4. Cloudflare: What is the aisuru-kimwolf botnet? (2026), https://www.cloudflare.com/learning/ddos/glossary/aisuru-kimwolf-botnet/, accessed: 2026-07-19
5. Djama, A., Maazouz, M., Kheddar, H., Rahim, M., Bengherbia, B.: High performance detection of ddos attacks with hybrid cnn-lstm architecture. In: 2025 International Conference on Intelligent Computer Systems, Data Science and Applications (IC2SDA). pp. 1–6 (2025). https://doi.org/10.1109/IC2SDA68097.2025.11331388
6. Draper-Gil, G., Lashkari, A.H., Mamun, M.S.I., A. Ghorbani, A.: Characterization of encrypted and vpn traffic using time-related features. In: Proceedings of the 2nd International Conference on Information Systems Security and Privacy - ICISSP. pp. 407–414. INSTICC, SciTePress (2016). https://doi.org/10.5220/0005740704070414
7. Farasat, T., Kim, J., Posegga, J.: Advancing network security: A comprehensive testbed and dataset for machine learning-based intrusion detection (2024), https://arxiv.org/abs/2410.18332
8. Farasat, T., Kim, J., Posegga, J.: Smartx intelligent sec: A security framework based on machine learning and ebpf/xdp (2024), https://arxiv.org/abs/2410.20244
9. Ghani, E.P., Sofwan, A., Somantri, M.: Ai-driven network security: Detecting and mitigating ddos, malware, and backdoor attacks with isolation and random forest algorithm. In: 2025 International Conference on Smart Computing, IoT and Machine Learning (SIML). pp. 1–6 (2025). https://doi.org/10.1109/SIML65326.2025.11080951
10. gRPC Authors: Introduction to grpc (05 2026), https://grpc.io/docs/what-is-grpc/introduction/, accessed: 2026-05-05

11. Heigl, M., Anand, K.A., Urmann, A., Fiala, D., Schramm, M., Hable, R.: On the improvement of the isolation forest algorithm for outlier detection with streaming data. Electronics **10**(13) (2021). https://doi.org/10.3390/electronics10131534, https://www.mdpi.com/2079-9292/10/13/1534
12. Høiland-Jørgensen, T., Brouer, J.D., Borkmann, D., Fastabend, J., Herbert, T., Ahern, D., Miller, D.: The eXpress data path: fast programmable packet processing in the operating system kernel. In: Proceedings of the 14th International Conference on Emerging Networking EXperiments and Technologies. p. 54–66. CoNEXT '18, Association for Computing Machinery, New York, NY, USA (2018). https://doi.org/10.1145/3281411.3281443
13. Lashkari, A.H., Gil, G.D., Mamun, M.S.I., Ghorbani, A.A.: Characterization of Tor traffic using time based features. In: International conference on information systems security and privacy. vol. 2, pp. 253–262. SciTePress (2017)
14. Liu, F.T., Ting, K.M., Zhou, Z.H.: Isolation forest. In: 2008 Eighth IEEE International Conference on Data Mining. pp. 413–422 (2008). https://doi.org/10.1109/ICDM.2008.17
15. Popić, S., Pezer, D., Mrazovac, B., Teslić, N.: Performance evaluation of using protocol buffers in the internet of things communication. In: 2016 International Conference on Smart Systems and Technologies (SST). pp. 261–265 (2016). https://doi.org/10.1109/SST.2016.7765670
16. Raspberry Pi: Raspberry pi 5 (05 2026), https://www.raspberrypi.com/products/raspberry-pi-5/, accessed: 2026-05-05
17. Reis, M., Serodio, C.: Edge ai for real-time anomaly detection in smart homes. Future Internet **17**, 179 (04 2025). https://doi.org/10.3390/fi17040179
18. Shaaban, A.R., Abd-Elwanis, E., Hussein, M.: Ddos attack detection and classification via convolutional neural network (cnn). In: 2019 Ninth International Conference on Intelligent Computing and Information Systems (ICICIS). pp. 233–238 (2019). https://doi.org/10.1109/ICICIS46948.2019.9014826
19. Sharafaldin, I., Lashkari, A.H., Hakak, S., Ghorbani, A.A.: Developing realistic distributed denial of service (ddos) attack dataset and taxonomy. In: 2019 International Carnahan Conference on Security Technology (ICCST). pp. 1–8 (2019). https://doi.org/10.1109/CCST.2019.8888419
20. Shurman, M., Khrais, R., Yateem, A.: Dos and ddos attack detection using deep learning and ids. International Arab Journal of Information Technology **17**, 655–661 (07 2020). https://doi.org/10.34028/iajit/17/4A/10
21. Vinothina, V., Prakash, V., Revathi, K., Kumar, C.S., Karthikeyen, N., Prathap, G.: An efficient DDoS attack detection scheme using autoencoder based on feature selection approaches. In: 2025 Third International Conference on Augmented Intelligence and Sustainable Systems (ICAISS). pp. 372–377. IEEE (2025)
22. Wickman, A., Rygaard, D.: Scalable Anomaly-Based Network Intrusion Detection Using a Statistical Model and Data Sketches. Master's thesis, Chalmers University of Technology (2025)

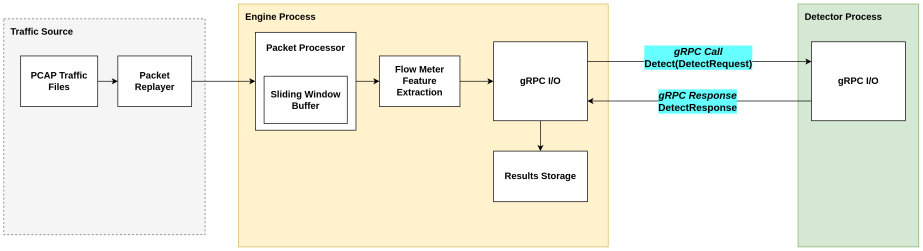
Appendix



(a) Monolithic



(b) Kafka-based



(c) gRPC-based

Fig. 1. Evaluated NIDS architectures. The detection pipeline is unchanged; only the communication boundary between the engine and detector differs.

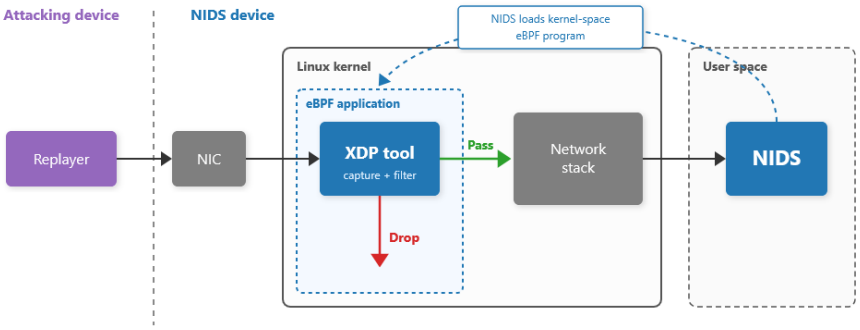


Fig. 2. Integration of flow-level detection with eBPF/XDP packet filtering.